



Gorgo Go for Java Programmers

May 31, 2026

Contents

11 Synchronization	1
The Go Memory Model	1
sync.Mutex and sync.RWMutex	1
sync.WaitGroup	3
sync.Once	4
sync.Cond	5
sync/atomic	7
The Race Detector	8
Key Points	9
Exercises	9

Chapter 11

Synchronization

Chapter 10 introduced goroutines and channels — Go’s preferred way to share work. But channels are not always the right tool: sometimes you need to protect a shared data structure, initialize something exactly once, or coordinate goroutines that don’t exchange messages. The `sync` package and `sync/atomic` cover those cases, offering primitives that Java programmers will recognize under new names.

The Go Memory Model

Before reaching for a mutex, you need to understand what the Go memory model guarantees. The model defines **happens-before** relationships: when a write in one goroutine is guaranteed to be visible to a read in another.

Without synchronization, goroutines are allowed to observe memory in any order. The compiler and CPU can reorder instructions as long as the reordering is invisible within a single goroutine — but that reordering is visible across goroutines.

The key rules:

- A send on a channel happens-before the corresponding receive from that channel.
- A `sync.Mutex.Unlock` happens-before any subsequent `Lock` that succeeds.
- `sync.Once.Do` completion happens-before any call to `Do` returns.
- Program initialization (all `init` functions) happens-before `main`.



Wut: Java programmers expect that writing a variable in one thread and reading it in another is safe as long as no two threads write at the same time. In Go (and in Java under the JMM) that is **not** safe unless there is a synchronization action between the write and the read. Without one, the reading goroutine may see a stale or partially written value.

The practical rule: any time two goroutines access the same memory and at least one of them is writing, you must use a channel, a mutex, or an atomic to establish a happens-before relationship. The race detector (`go test -race`) enforces this mechanically — see the end of this chapter.

`sync.Mutex` and `sync.RWMutex`

`sync.Mutex` is Go’s equivalent of Java’s synchronized block. It provides exclusive access to a critical section.

```
import "sync"
```

```
type Playlist struct {
```

```

    mu    sync.Mutex // protects tracks
    tracks []string
}

func (p *Playlist) Add(track string) {
    p.mu.Lock()
    defer p.mu.Unlock()
    p.tracks = append(p.tracks, track)
}

func (p *Playlist) Tracks() []string {
    p.mu.Lock()
    defer p.mu.Unlock()
    result := make([]string, len(p.tracks))
    copy(result, p.tracks)
    return result
}

```

Lock blocks until the mutex is available. Unlock releases it. Using `defer p.mu.Unlock()` immediately after Lock ensures the mutex is always released, even if the function panics.



Tip: Always use `defer mu.Unlock()` on the very next line after `mu.Lock()`. This eliminates the risk of forgetting to unlock on every return path.



Trap: A `sync.Mutex` must not be copied after first use. If you embed one in a struct, always pass the struct by pointer (`*Playlist`), never by value. Copying a locked mutex is undefined behavior. [*pointer-receiver-for-mutex*]

Comparison with Java synchronized

Java's `synchronized` keyword takes an **object monitor** as its lock:

```

synchronized (this) {
    tracks.add(track);
}

```

Go has no per-object monitor. You declare an explicit `sync.Mutex` field and lock it by name. This is more verbose but also more precise: you can have multiple independent mutexes protecting different fields in the same struct.

sync.RWMutex

`sync.RWMutex` extends `sync.Mutex` with separate read and write locks. Multiple goroutines can hold the read lock simultaneously; the write lock requires exclusive access. Use it when reads far outnumber writes.

```

type Catalog struct {
    mu    sync.RWMutex
    songs map[string]string // title -> artist
}

func (c *Catalog) Lookup(title string) (string, bool) {
    c.mu.RLock()           // multiple goroutines can hold RLock at once
    defer c.mu.RUnlock()
    artist, ok := c.songs[title]
}

```

```

    return artist, ok
}

func (c *Catalog) Add(title, artist string) {
    c.mu.Lock()           // exclusive write lock
    defer c.mu.Unlock()
    c.songs[title] = artist
}

```

RLock and RUnlock are the read-side pair. Lock and Unlock are the write-side pair, identical to sync.Mutex.



Tip: Only reach for RWMutex when you have measured a contention problem. A plain Mutex is faster for workloads with balanced reads and writes because RWMutex has higher overhead to track readers.

sync.WaitGroup

sync.WaitGroup lets a goroutine wait for a collection of other goroutines to finish. Java programmers typically use CountdownLatch or CompletableFuture.allOf for the same pattern; WaitGroup is simpler.

```

package main

import (
    "fmt"
    "sync"
)

func main() {
    songs := []string{"Ditto", "Hype Boy", "ETA"} // NewJeans
    var wg sync.WaitGroup

    for _, song := range songs {
        wg.Add(1)
        go func(s string) {
            defer wg.Done()
            fmt.Printf("playing: %s\n", s)
        }(song)
    }

    wg.Wait() // blocks until all three goroutines call Done
    fmt.Println("all songs finished")
}

```

The three operations are:

```

wg.Add(n) // increment the counter by n --- call before launching goroutines
wg.Done() // decrement the counter by 1 --- call when a goroutine finishes
wg.Wait() // block until the counter reaches zero

```



Trap: Call wg.Add(n) **before** launching the goroutines, not inside them. If the goroutine calls Add and the main goroutine calls Wait before the goroutine starts, Wait will return immediately with a zero counter.



Trap: Pass the `WaitGroup` by pointer or use a closure that captures it. Copying a `WaitGroup` after first use is a bug.

Fan-out / Fan-in Without Channels

`WaitGroup` is the idiomatic way to fire off *N* goroutines and wait for all of them without the ceremony of a results channel. If you need return values, combine `WaitGroup` with a pre-allocated slice (one slot per goroutine, no contention) or use `errgroup` from `golang.org/x/sync` (covered in Chapter 12).

`sync.Once`

`sync.Once` runs a function exactly once, no matter how many goroutines call it concurrently. It is the safe, idiomatic replacement for the double-checked locking pattern that Java programmers used before `volatile` was fixed in Java 5.

```
package main

import (
    "fmt"
    "sync"
)

var (
    once      sync.Once
    catalog   map[string]string
)

func loadCatalog() {
    once.Do(func() {
        // expensive initialization runs exactly once
        catalog = map[string]string{
            "Ditto":    "NewJeans",
            "Dynamite": "BTS",
            "Butter":   "BTS",
        }
        fmt.Println("catalog loaded")
    })
}

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 5; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            loadCatalog()
        }()
    }
    wg.Wait()
    fmt.Println(catalog["Dynamite"]) // BTS
}
```


Output:

```
catalog loaded
BTS
```

The message “catalog loaded” appears exactly once even though five goroutines called `loadCatalog` concurrently. Subsequent calls to `once.Do` return immediately.



Wut: `sync.Once` caches the **first** call’s completion, not its result. If the function passed to `Do` panics, the `once` is still considered done — subsequent callers see the (partial) side effects and `Do` never runs again. Panic inside `Do` is almost always a bug.

The Java equivalent that `sync.Once` replaces:

```
// Java double-checked locking (error-prone before Java 5)
private volatile Map<String,String> catalog;

public Map<String,String> getCatalog() {
    if (catalog == null) {
        synchronized (this) {
            if (catalog == null) {
                catalog = loadExpensive();
            }
        }
    }
    return catalog;
}
```

Go’s `sync.Once` does this correctly with zero boilerplate.

sync.Cond

`sync.Cond` is a condition variable — a way for goroutines to wait for a predicate to become true and for other goroutines to signal when state changes. Java’s equivalent is `Object.wait()` / `Object.notify()` / `Object.notifyAll()`, or the more modern `java.util.concurrent.locks.Condition`.

A `sync.Cond` is always associated with a `sync.Locker` (usually a `*sync.Mutex`):

```
cond := sync.NewCond(&mu) // create a Cond associated with mu
```

The three operations are:

```
cond.Wait()    // atomically unlock mu and suspend; re-lock mu on wake
cond.Signal()  // wake one waiting goroutine
cond.Broadcast() // wake all waiting goroutines
```

Here is a producer/consumer example with a bounded queue:

```
package main

import (
    "fmt"
    "sync"
)

type Queue struct {
    mu    sync.Mutex
    cond  *sync.Cond
}
```

```

    items []string
    cap    int
}

func NewQueue(cap int) *Queue {
    q := &Queue{cap: cap}
    q.cond = sync.NewCond(&q.mu)
    return q
}

func (q *Queue) Push(item string) {
    q.mu.Lock()
    for len(q.items) == q.cap {
        q.cond.Wait() // releases lock, waits for signal, re-acquires lock
    }
    q.items = append(q.items, item)
    q.cond.Broadcast()
    q.mu.Unlock()
}

func (q *Queue) Pop() string {
    q.mu.Lock()
    for len(q.items) == 0 {
        q.cond.Wait()
    }
    item := q.items[0]
    q.items = q.items[1:]
    q.cond.Broadcast()
    q.mu.Unlock()
    return item
}

func main() {
    q := NewQueue(2)

    go func() {
        for _, song := range []string{"Hype Boy", "ETA", "Ditto"} {
            q.Push(song)
            fmt.Println("pushed:", song)
        }
    }()

    for i := 0; i < 3; i++ {
        fmt.Println("popped:", q.Pop())
    }
}

```



Trap: Always check the wait condition in a for loop, not an if. A goroutine can wake spuriously (or because another goroutine consumed the item first), so you must re-check the predicate before proceeding. This is the same rule as Java's `while (condition) { lock.wait(); }`.



Tip: Prefer Broadcast over Signal when multiple goroutines could each satisfy the predicate. A spurious Signal that wakes the wrong waiter wastes a cycle; Broadcast wakes them all and lets them re-check.



Wut: `sync.Cond` cannot be used with Go's `select` statement. If you need to select between "condition is met" and a timeout or another channel, restructure using channels instead.

sync/atomic

The `sync/atomic` package provides low-level atomic memory operations. In Go 1.19, typed atomic types were added that are much safer than the old function-based API.

Typed Atomics (Go 1.19+)

```
import "sync/atomic"
```

The typed atomic types:

```
atomic.Bool           // atomic bool
atomic.Int32, atomic.Int64 // atomic signed integers
atomic.Uint32, atomic.Uint64 // atomic unsigned integers
atomic.Uintptr        // atomic uintptr
atomic.Pointer[T]     // atomic pointer to T (generic, Go 1.19+)
```

Each type provides the same set of methods:

```
func (x *Int64) Load() int64           // read atomically
func (x *Int64) Store(val int64)        // write atomically
func (x *Int64) Add(delta int64) (new int64) // add and return new value
func (x *Int64) Swap(new int64) (old int64) // set and return old value
func (x *Int64) CompareAndSwap(old, new int64) bool // CAS: swap if current == old
```

`atomic.Pointer[T]` uses generics so you get type safety without a cast:

```
func (x *Pointer[T]) Load() *T           // load atomically
func (x *Pointer[T]) Store(val *T)        // store atomically
func (x *Pointer[T]) Swap(new *T) *T      // swap atomically
func (x *Pointer[T]) CompareAndSwap(old, new *T) bool // CAS
```

Here is a play-count tracker using atomic integers:

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)

func main() {
    var plays atomic.Int64
    var wg sync.WaitGroup

    songs := []string{"Dynamite", "Butter", "Ditto", "Hype Boy", "ETA"}
```

```

for _, song := range songs {
    wg.Add(1)
    go func(s string) {
        defer wg.Done()
        plays.Add(1) // no mutex needed
        fmt.Printf("played: %s\n", s)
    }(song)
}

wg.Wait()
fmt.Println("total plays:", plays.Load()) // 5
}

```

When to Use Atomics vs Mutexes

Use atomics for:

- Simple counters (hits, errors, bytes transferred).
- A single flag that goroutines read and one goroutine writes.
- Lock-free data structures where you understand the ABA problem.

Use a mutex when:

- You are protecting more than one variable together (invariant maintenance).
- The critical section does more than a single load/store/add.



Tip: Java's `java.util.concurrent.atomic.AtomicLong` maps directly to `atomic.Int64`. The semantics are the same: sequential consistency for the atomic variable itself, no guarantee about other memory.

The Race Detector

Go ships a built-in race detector based on ThreadSanitizer. Enable it with the `-race` flag:

```

go test -race ./...
go run -race main.go
go build -race -o myapp .

```

When a data race is detected at runtime, the race detector prints a detailed report showing both the racing accesses and their goroutine stack traces:

```

=====
WARNING: DATA RACE
Write at 0x00c0000b4010 by goroutine 7:
    main.main.func1()
        /tmp/race.go:12 +0x2c

Previous read at 0x00c0000b4010 by goroutine 6:
    main.main.func1()
        /tmp/race.go:9 +0x30
=====

```

The race detector adds roughly 5–10x runtime overhead and 2–3x memory overhead. It is not suitable for production, but it should run in CI on every pull request and on every test suite.



Tip: Run `go test -race ./...` in CI on every push. Data races are undefined behavior: the program may produce wrong results, crash, or appear to work correctly on your machine while failing in production. The race detector is the only reliable way to find them.

Here is an example the race detector catches immediately:

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    counter := 0 // shared without synchronization
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            counter++ // DATA RACE: concurrent read-modify-write
        }()
    }
    wg.Wait()
    fmt.Println(counter) // result is unpredictable
}
```

The fix: replace counter with `atomic.Int64` or protect it with a `sync.Mutex`.

Key Points

- The Go memory model defines **happens-before** relationships; without synchronization, goroutines may observe stale memory.
- `sync.Mutex` provides exclusive access; use `defer mu.Unlock()` immediately after `mu.Lock()`.
- `sync.RWMutex` allows concurrent readers; reach for it only when reads dominate and you have measured a contention problem.
- `sync.WaitGroup` is the idiomatic fan-out/fan-in primitive when you do not need to pass results back through a channel.
- `sync.Once` runs a function exactly once; it is the correct, simple replacement for double-checked locking.
- `sync.Cond` is a condition variable; always check the predicate in a `for` loop, not an `if`.
- `atomic.Int64`, `atomic.Pointer[T]`, and friends (Go 1.19+) are the type-safe atomic primitives; use them for simple counters and flags.
- Run `go test -race ./...` in CI; a data race is undefined behavior and may be silent on your laptop.

Exercises

1. **Think about it:** Java's `synchronized` keyword locks an object's monitor, which is built into every Java object. Go has no per-object monitor; instead you declare explicit `sync.Mutex` fields. What are the practical advantages and disadvantages of each approach? Consider: what happens when you need to protect two independent fields in the same struct, and how would you do it with each language's mechanism?
2. **What does this print?**

```

package main

import (
    "fmt"
    "sync"
)

func main() {
    var once sync.Once
    var wg sync.WaitGroup
    results := make([]string, 3)

    for i := 0; i < 3; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            once.Do(func() {
                results[n] = "loaded"
            })
            if results[n] == "" {
                results[n] = "skipped"
            }
        }(i)
    }

    wg.Wait()
    loaded := 0
    skipped := 0
    for _, r := range results {
        if r == "loaded" {
            loaded++
        } else if r == "skipped" {
            skipped++
        }
    }
    fmt.Printf("loaded=%d skipped=%d\n", loaded, skipped)
}

```

3. **Calculation:** Consider the following program fragment:

```

var counter atomic.Int64
var wg sync.WaitGroup

for i := 0; i < 4; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        counter.Add(10)
    }()
}
wg.Wait()
fmt.Println(counter.Load())

```

- (a) What value does `counter.Load()` always print, regardless of goroutine scheduling order?
- (b) If `counter.Add(10)` were replaced by `counter.Add(int64(i))` (capturing `i` from the loop), what

bug would be introduced, and what range of printed values would be possible?

4. Where is the bug?

```
package main

import (
    "fmt"
    "sync"
)

type SafeMap struct {
    mu sync.Mutex
    m  map[string]int
}

func NewSafeMap() SafeMap {
    return SafeMap{m: make(map[string]int)}
}

func (s SafeMap) Inc(key string) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.m[key]++
}

func (s SafeMap) Get(key string) int {
    s.mu.Lock()
    defer s.mu.Unlock()
    return s.m[key]
}

func main() {
    sm := NewSafeMap()
    var wg sync.WaitGroup
    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            sm.Inc("Butter")
        }()
    }
    wg.Wait()
    fmt.Println(sm.Get("Butter"))
}
```

5. **Write a program:** Implement a concurrent-safe `RateLimiter` struct that uses a `sync.Mutex` to protect a counter and a `time.Time` field tracking when the window resets. The struct should have a method `Allow(n int) bool` that returns true if `n` tokens are available in the current one-second window, deducting them if so, and false otherwise (without deducting). Write a `main` function that launches 10 goroutines, each calling `Allow(1)` in a loop 5 times, and prints how many calls were allowed versus denied across all goroutines combined. Use `sync.WaitGroup` to wait for all goroutines to finish.

